Clickhouse DB

Introduction to PostgreSQL + TimescaleDB

PostgreSQL is a powerful, open-source relational database system. It's known for its extensibility, SQL compliance, and robustness, making it a popular choice for general-purpose data storage.

TimescaleDB, an extension built on top of PostgreSQL, is designed specifically for managing **timeseries data** at scale. It combines the powerful relational capabilities of PostgreSQL with optimizations for handling large amounts of time-ordered data. It provides efficient data storage, retrieval, and analysis for applications like **IoT**, **finance**, **logs**, and other time-sensitive data.

Key Features of PostgreSQL with TimescaleDB:

- **Relational model**: TimescaleDB keeps all the relational features of PostgreSQL, such as support for joins, foreign keys, and complex SQL queries, while optimizing time-based data.
- **Hypertables**: TimescaleDB partitions your data into chunks based on time. A **hypertable** is a logical abstraction that simplifies working with partitioned time-series data.
- **Chunks**: A **chunk** in TimescaleDB is a segment of data (typically based on a time interval like a day, week, or month) that is physically stored on disk. Each hypertable is divided into chunks for efficient data storage and query execution. This enables TimescaleDB to quickly access data within time-based partitions.
- **Compression**: TimescaleDB can automatically compress older data, reducing storage costs without sacrificing query performance.
- **Real-time ingestion**: TimescaleDB is optimized for high-throughput writes of timeseries data, making it good for applications with high write loads, such as monitoring or IoT.
- **Scalability**: TimescaleDB supports both vertical scaling (more powerful hardware) and horizontal scaling (through sharding).
- **SQL support**: Full support for SQL queries, ACID transactions, and complex aggregations.

Data Storage in TimescaleDB

TimescaleDB is an extension of **PostgreSQL** designed specifically for handling timeseries data efficiently. The key feature in TimescaleDB is its ability to partition timeseries data into **chunks**.

How Data is Stored in TimescaleDB (Chunks)

- **Hypertables**: A **hypertable** is the main abstraction in TimescaleDB. It behaves like a normal PostgreSQL table but is optimized for timeseries data. A hypertable is made up of one or more **chunks**. The main goal of hypertables is to efficiently store and query large datasets, especially when the data is time-ordered.
  - **Chunks**: A chunk is a **physical partition** of data based on a time interval. For example, if you configure TimescaleDB to partition data by month, each chunk could represent data from a single month.
  - **Automatic Partitioning**: TimescaleDB automatically partitions data when you create a hypertable. By default, it partitions based on the `timestamp` column (but can be configured to partition on other columns too). This partitioning ensures that only relevant chunks are scanned during queries, significantly improving performance when dealing with large amounts of data.
  - **Chunk Management**:
    - When data is inserted into a hypertable, it is placed into the appropriate chunk based on its timestamp.

    - Over time, chunks can grow or be dropped (e.g., if you set a retention policy).
    - TimescaleDB allows compression for older chunks, which reduces storage requirements without sacrificing query performance.
  - **TimescaleDB** (and PostgreSQL in general) uses **row-based storage**, where each row contains data for all columns. Even if you're querying only a few columns, it still has to scan the entire row, which can result in higher I/O and slower performance for large datasets.
- **Data Example**:
  Suppose you have a table `measurements` with columns `timestamp`, `user_id`, `meter_id`, and `value`. When you create a hypertable on this table, TimescaleDB will partition the data into chunks based on `timestamp`. If you set the partitioning interval to `1 month`, each chunk will contain the data for one month, and queries will only need to scan the chunks that match the time range.
- **Benefits of Chunks**:
  - **Efficient Querying**: Only the relevant chunks are queried based on the time filter. For example, if your query is only for January 2021 data, TimescaleDB will only scan the chunk that contains January 2021 data.
  - **Data Compression**: Older chunks can be compressed, reducing storage usage.
  - **Retention Policies**: You can set retention policies to automatically drop old chunks that are no longer needed.

How Data is Partitioned in TimescaleDB:

- Chunks are divided based on time intervals (e.g., day, week, month), and each chunk is treated as a separate table in the database.
- For example, if you have a dataset spanning multiple years, you could have a chunk for every month. Queries that filter by time only need to scan the relevant chunks, which improves query performance significantly.

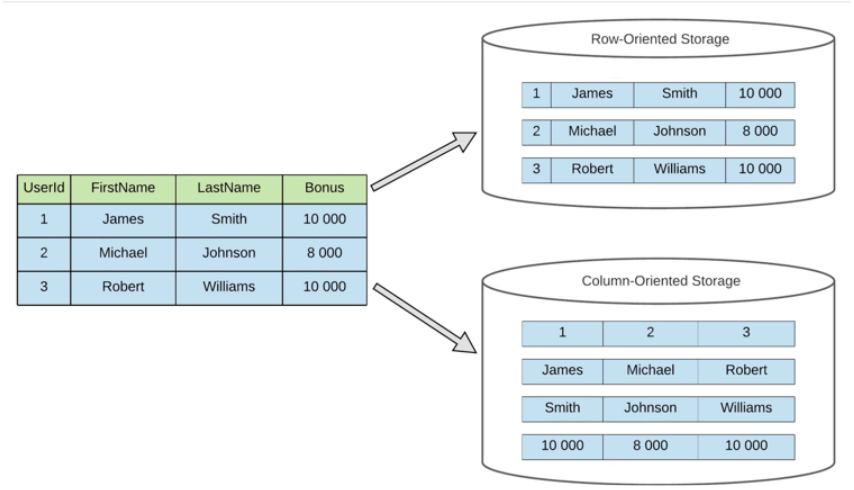
Introduction to ClickHouse

ClickHouse is an open-source columnar database management system designed for high-performance **OLAP (Online Analytical Processing)** workloads. It excels at processing large volumes of data and is particularly optimized for analytics on large datasets.

Key Features of ClickHouse:

- **Columnar storage**: ClickHouse stores data in columns rather than rows. This makes it very efficient for analytical queries where only a subset of columns need to be scanned, as opposed to reading entire rows of data.
- In **columnar databases**, each column of data is stored separately. This allows ClickHouse to scan only the columns relevant to the query, making it significantly more efficient for **analytical queries** that only require a subset of columns.
- For example, in a **timeseries query** where you're only aggregating or filtering by `timestamp` and `value` columns, ClickHouse doesn't need to read data from all columns (like `user_id` or `meter_id`) as it would in a row-based database. This is a **huge performance advantage** for queries that involve large datasets but only need specific columns.
- **Real-time analytics**: ClickHouse is designed to handle fast data ingestion and real-time query processing, making it ideal for systems that require up-to-the-minute analysis of vast amounts of data (e.g., monitoring, log aggregation).
- **High performance**: It is built for fast aggregation, filtering, and joining over large datasets. ClickHouse can perform queries at very high speed, even on massive amounts of data.

- **Scalability**: ClickHouse is designed to scale horizontally across multiple machines, which allows it to handle petabytes of data.
- **Parallel Query Execution**:
  - **ClickHouse** has **massively parallel processing** (MPP) capabilities, meaning that queries can be distributed across multiple nodes and executed in parallel. This parallelism is a big reason why ClickHouse is able to handle large analytical queries at high speed.
- **Time-series support**: Like TimescaleDB, ClickHouse is optimized for handling timeseries data. It supports partitioning by time, efficient inserts, and querying over large time ranges.
- **SQL interface**: ClickHouse uses an SQL-like query language for data access, making it accessible to users familiar with relational databases.



Data Storage in ClickHouse

ClickHouse is a columnar database optimized for analytical workloads, specifically designed for high-speed OLAP (Online Analytical Processing). It uses a different approach to partitioning data compared to TimescaleDB.

How Data is Stored in ClickHouse (Partitions)

- **MergeTree Engine**: ClickHouse uses the **MergeTree** family of table engines, which are the most common engine used for partitioned data in ClickHouse. The MergeTree engine allows for efficient storage and querying of large datasets. Within the MergeTree engine, data is partitioned into **partitions** and **parts**.
  - **Partitions**: Data in ClickHouse is divided into **partitions** based on a key (usually the `timestamp` column for timeseries data). Each partition typically contains data for a specific time range (e.g., a day, a week, or a month). Partitions are stored as separate files on disk.
  - **Parts**: Within each partition, data is stored in **parts**. A part is a chunk of data that represents a small subset of the partition and can be merged with other parts over time. The merging process helps optimize storage and query performance by combining smaller parts into larger ones as data ages.
- **Data Example**:
  If you are storing timeseries data in ClickHouse, you would typically partition your table by the `timestamp` column. For instance, if you're using daily partitions, ClickHouse would store the data for each day in a separate partition. Over time, these partitions are merged to optimize space and query performance.
- **Partitioning Scheme in ClickHouse**:

- **Example Partitioning by Date**: You might configure a table in ClickHouse to partition data by `toStartOfDay(timestamp)` (i.e., daily partitions).
- **Merge and Optimize**: ClickHouse uses a **background merging** process where small data parts within a partition are periodically merged to reduce overhead and improve performance. This is similar to TimescaleDB's chunk management, but ClickHouse's merging process happens automatically in the background.

Benefits of Partitions in ClickHouse:

- **Efficient Querying**: ClickHouse uses **partition pruning**, which means that only the relevant partitions are scanned during queries. If you're querying data for January 2021, only the January 2021 partition will be scanned, improving performance.
- **Data Merging**: As data ages, smaller parts within a partition are merged into larger parts, optimizing storage and query performance.
- **Scalability**: ClickHouse partitions data by time and stores these partitions across distributed systems, enabling it to scale horizontally very efficiently.

Key Differences Between PostgreSQL + TimescaleDB and ClickHouse

1. Data Model and Schema Design

- **PostgreSQL with TimescaleDB**:
  - **Relational Schema**: TimescaleDB builds on PostgreSQL's relational schema, which is perfect for situations where you need to combine timeseries data with other types of data (like relational data).
  - **Hypertables and Chunks**: Data in TimescaleDB is stored in **hypertables**, which are logical representations of partitioned tables. Hypertables are broken into **chunks** based on time intervals (e.g., daily, weekly). Each chunk is stored as a separate table, and TimescaleDB manages which chunk data should be stored in for efficient query processing.
  - **Use Case**: Well-suited for applications where time-series data interacts with relational data, and queries require complex joins, aggregations, and transactional consistency (e.g., IoT data with device metadata, financial data, logs with metadata).
- **ClickHouse**:
  - **Columnar Storage**: Data in ClickHouse is stored in columns, which is ideal for **analytic queries** where you need to filter and aggregate data on specific columns (e.g., time and sensor readings).
  - **Partitioning by Time**: ClickHouse can partition data by time intervals, similar to TimescaleDB. However, ClickHouse uses more manual configurations for partitioning, and it typically does not involve complex relational joins.
  - **Use Case**: Best suited for high-performance analytics and aggregation on large timeseries datasets (e.g., real-time dashboards, monitoring, and log analysis).

2. Performance and Scalability

- **PostgreSQL with TimescaleDB**:
  - **Performance**: TimescaleDB's performance is optimized for timeseries workloads, but since it is built on top of PostgreSQL, its performance is limited by PostgreSQL's overhead. While TimescaleDB handles time-series data more efficiently than a regular PostgreSQL database, it may still face challenges when working with very large datasets at scale.
  - **Chunks for Scalability**: TimescaleDB manages the scale of time-series data by splitting it into **chunks**(partitions), making queries and data management more efficient by only scanning relevant partitions.

- **Scalability**: TimescaleDB can scale vertically (on a single machine) or horizontally (by sharding across multiple nodes), but setting up horizontal scaling is more complex than with ClickHouse.

- **ClickHouse**:
  - **Performance**: ClickHouse is **exceptionally fast** at analytical queries, especially for aggregations and filtering over large datasets. It is optimized for read-heavy workloads and can handle billions of rows of data very efficiently.
  - **Real-time Analytics**: Due to its **columnar storage** and optimized query engine, ClickHouse is ideal for real-time analytics over large volumes of data.
  - **Scalability**: ClickHouse was built with **horizontal scalability** in mind. It can easily scale across a cluster of nodes, which allows it to handle massive datasets (terabytes or petabytes of data).

3. Real-time Data Ingestion and Analytics

- **PostgreSQL with TimescaleDB**:
  - **Real-time ingestion**: TimescaleDB is capable of handling **real-time data ingestion**, especially with high throughput for timeseries data. However, if the dataset is very large or if real-time analytics is highly demanding, its performance might lag behind ClickHouse.
  - **Query Speed**: TimescaleDB works well with moderate-sized datasets and typical timeseries use cases but can face performance bottlenecks with large, highly concurrent workloads or when large time windows are queried.
- **ClickHouse**:
  - **Real-time ingestion**: ClickHouse is built for **high-throughput data ingestion**. It can ingest millions of rows per second and is optimized for real-time analytics on timeseries data.
  - **Query Speed**: ClickHouse is designed for fast aggregation and filtering, making it much faster than TimescaleDB for large-scale real-time analytics, especially when querying vast amounts of data.
  - Designed for append only data, not UPSERT

4. Data Relationships and Query Complexity

- **PostgreSQL with TimescaleDB**:
  - **Complex Queries**: TimescaleDB benefits from the full power of **SQL** and supports **complex queries**, including joins, subqueries, and advanced relational data operations.
  - **Hybrid Workloads**: If you need to perform complex queries involving both timeseries data and other relational data (e.g., joining timeseries with metadata), TimescaleDB is a better choice.
- **ClickHouse**:
  - **Simpler Queries**: While ClickHouse supports SQL-like queries, it is better suited for analytical queries that do not require complex joins or relational operations. It's optimized for performing large-scale aggregations and filtering rather than complex joins.
  - **Use Case**: Ideal for large-scale analytics and dashboards, but not as suitable for scenarios that require complex relationships between timeseries data and other types of data.

5. Maintenance and Operational Complexity

- **PostgreSQL with TimescaleDB**:
  - **Maintenance**: TimescaleDB benefits from the maturity and operational stability of PostgreSQL. It is easier to set up if you are already familiar with PostgreSQL, and it supports a variety of management tools.
  - **Operational Complexity**: Horizontal scaling (sharding) in TimescaleDB can be complex, and managing large datasets may require careful tuning.
- **ClickHouse**:

- **Maintenance**: ClickHouse is relatively easy to set up but managing large clusters can be more involved, especially if you need to distribute the data across multiple nodes.
- **Operational Complexity**: ClickHouse is optimized for large-scale deployments but may require more infrastructure and monitoring expertise to manage clusters effectively.

Comparison of Data Storage: TimescaleDB vs. ClickHouse

Feature	TimescaleDB	ClickHouse
Data Model	Relational (uses hypertables, chunks)	Columnar (uses MergeTree engine, partitions)
Partitioning	Chunks (partitioned by time, stored as separate tables)	Partitions (based on time, stored in files)
Chunking/Part Merging	Automatic chunk creation based on time intervals	Data is stored in parts, which are merged in the background
Storage Engine	Built on PostgreSQL, with timeseries optimizations	Columnar storage engine optimized for OLAP
Compression	Supports compression for older chunks	Automatic background merging optimizes storage
Scalability	Vertical and horizontal scaling (sharding)	Horizontal scaling, highly distributed
Use Case	Best for hybrid transactional and analytical workloads (relational data + timeseries)	Best for pure timeseries data and high-speed OLAP queries

---

Testing Which Is Better for Use Case:

To decide which database is better for your specific use case, especially for **timeseries analytics**, you should perform a series of tests to measure **performance**, **scalability**, and **real-time ingestion** based on your workload.

Here are the **key tests** to help you compare **TimescaleDB** and **ClickHouse**:

1. Ingestion Speed

- **Test**: Measure the speed at which each database can ingest timeseries data.
  - **TimescaleDB**: Test how fast it can insert large volumes of timeseries data (e.g., millions of rows) and how it handles high-throughput ingestion.

- **ClickHouse**: Test ClickHouse's ability to handle high-speed data ingestion. ClickHouse is optimized for real-time analytics, so it should perform better with high-throughput workloads.
- **Measure**: The number of rows ingested per second and any impact on query performance as the data volume increases.

2. Query Performance (Range Queries)

- **Test**: Measure how quickly each database can run queries that filter by time.
  - **TimescaleDB**: Run queries that filter by a time range (e.g., querying data for a specific month). Measure the time it takes to fetch results from different time periods, considering the partitioning mechanism (chunks).
  - **ClickHouse**: Run similar queries on ClickHouse, ensuring that it uses partition pruning effectively.
  - **Measure**: Query execution time and resource consumption (CPU, disk I/O).

3. Complex Queries with Aggregations

- **Test**: Measure the performance of each database when running complex queries with aggregations (e.g., `AVG`, `SUM`, `COUNT`) over large timeseries datasets.
  - **TimescaleDB**: Test how well TimescaleDB performs with these types of queries on large, partitioned datasets.
  - **ClickHouse**: Since ClickHouse is optimized for OLAP queries, it should perform very well in this scenario.
  - **Measure**: Query execution time, especially for aggregations over large time ranges.

4. Scalability and Distributed Queries

- **Test**: Measure how well each system scales as the data volume increases.
  - **TimescaleDB**: Test TimescaleDB with sharding and check how it handles large datasets across multiple nodes.
  - **ClickHouse**: Test ClickHouse's ability to scale horizontally by adding nodes to the cluster.
  - **Measure**: Query performance and data ingestion rates as you scale up the data volume.

TESTING

On a dataset of 2M rows, we have conducted the same query both for timescaledb and for clickhouse.

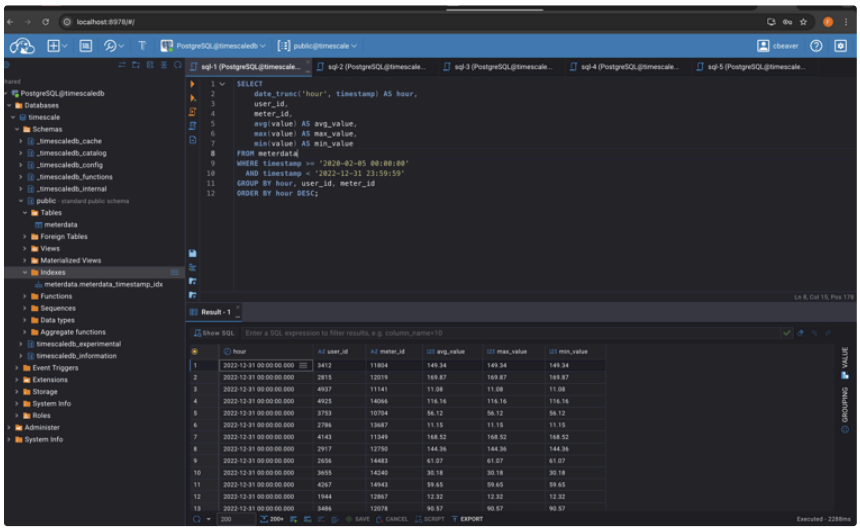
Dataset

```
timestamp,user_id,meter_id,value
2022-10-12 00:00:00,128,484,143.01
2020-03-03 00:00:00,113,406,162.73
2021-09-21 00:00:00,280,270,98.66
2022-02-07 00:00:00,273,238,67.68
2022-02-04 00:00:00,269,218,92.11
```

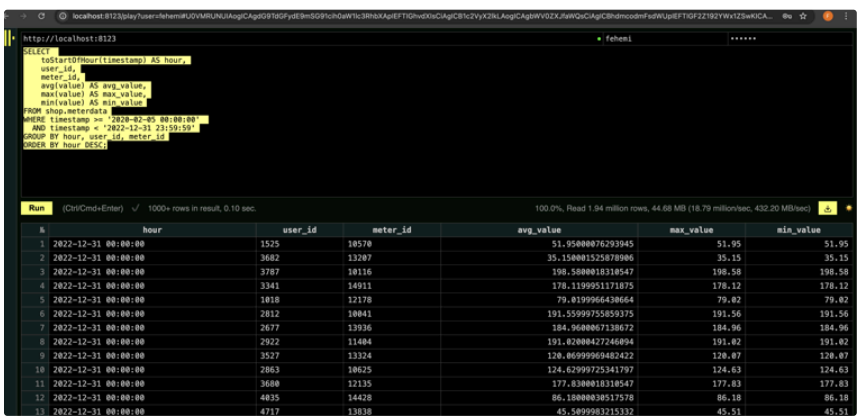
Timescaledb

```
  CREATE TABLE IF NOT EXISTS meterdata (
    timestamp TIMESTAMPTZ,
    user_id TEXT,
    meter_id TEXT,
    value FLOAT
  );

  -- Create hypertable for time-series data
  SELECT create_hypertable('meterdata', 'timestamp', chunk_time_interva

  -- Configure Compression
  ALTER TABLE meterdata SET (
    timescaledb.compress,
    timescaledb.compress_segmentby = 'meter_id',
    timescaledb.compress_orderby = 'timestamp'
);

    -- Load the CSV data into the table
```

```
COPY meterdata(timestamp, user_id, meter_id, value)
FROM '/csv-data/timeseries_data.csv' DELIMITER ',' CSV HEADER;

CREATE INDEX idx_meterdata_user_meter_ts
ON meterdata (user_id, meter_id, timestamp DESC);
```



Clickhouse



№	hour	user_id	meter_id	avg_value	max_value	min_value
1	2022-12-31 00:00:00	1525	10570	51.95000076293945	51.95	51.95
2	2022-12-31 00:00:00	3682	13207	35.15000152587906	35.15	35.15
3	2022-12-31 00:00:00	3787	10116	198.5800018310547	198.58	198.58
4	2022-12-31 00:00:00	3341	14911	178.1199951171875	178.12	178.12
5	2022-12-31 00:00:00	1018	12178	79.0199966430664	79.02	79.02
6	2022-12-31 00:00:00	2812	10041	191.55999755859375	191.56	191.56
7	2022-12-31 00:00:00	2677	13936	184.9600067138672	184.96	184.96
8	2022-12-31 00:00:00	2922	11404	191.02000427246094	191.02	191.02
9	2022-12-31 00:00:00	3527	13324	120.06999969482422	120.07	120.07
10	2022-12-31 00:00:00	2863	10625	124.62999725341797	124.63	124.63
11	2022-12-31 00:00:00	3680	12135	177.8300018310547	177.83	177.83
12	2022-12-31 00:00:00	4035	14428	86.18000030517578	86.18	86.18
13	2022-12-31 00:00:00	4717	13838	45.5099983215332	45.51	45.51